

NAME: Atchaya Vharsne S

REG NO.: 192524185

COURSE CODE: CSA0401

COURSE NAME: Operating Systems

CO3 AT1

1.Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address. Display appropriate messages for valid and invalid addresses.

```
main.c Run
1 #include <stdio.h>
2 #include <stdlib.h>
3 void translate_address(unsigned int logical_address, unsigned int base_register, unsigned int
   limit_register) {
4     printf("\n-----\n");
5     printf("Logical Address ")
6     if (logical_address < limit_register) {
7         unsigned int physical_address = base_register + logical_address;
8         printf("Status: VALID ADDRESS\n");
9         printf("Physical Address Computation: %u (Base) + %u (Logical)\n", base_register,
   logical_address);
10        printf("Translated Physical Address : %u\n", physical_address);
11    } else {
12        printf("Status: INVALID ADDRESS (Addressing Error / Segmentation Fault)\n");
13        printf("Error: Logical address %u exceeds or equals Limit limit of %u.\n", logical_address,
   limit_register);
14    }
15    printf("-----\n");
16 int main() {
17     unsigned int base_register, limit_register;
18     unsigned int logical_address;
19     printf("=== Memory Management Unit (MMU) Simulation ===\n\n");
20     printf("Enter Base Register value (starting physical address): ");
21     if (scanf("%u", &base_register) != 1) return 1;
22     printf("Enter Limit Register value (range/size of allocated memory): ");
23     if (scanf("%u", &limit_register) != 1) return 1;
24     char choice;
25     do {
26         printf("\nEnter Logical Address to translate: ");
27         if (scanf("%u", &logical_address) != 1) break;
28
29         translate_address(logical_address, base_register, limit_register);
30
31         printf("\nTranslate another address? (y/n): ");
32         scanf(" %c", &choice);
33     } while (choice == 'y' || choice == 'Y');
34     printf("\nSimulation terminated\n");
```

```
Output
=== Memory Management Unit (MMU) Simulation ===

Enter Base Register value (starting physical address): 3004
Enter Limit Register value (range/size of allocated memory): 1209

Enter Logical Address to translate: 346

-----
Logical Address Status: VALID ADDRESS
Physical Address Computation: 3004 (Base) + 346 (Logical)
Translated Physical Address : 3350
-----

Translate another address? (y/n): y

Enter Logical Address to translate: 1209
:RROR!

-----
Logical Address Status: INVALID ADDRESS (Addressing Error / Segmentation Fault)
Error: Logical address 1209 exceeds or equals Limit limit of 1209.
-----

Translate another address? (y/n): |
```

How Address Translation Works

- **Limit Check:** The MMU verifies that $\text{Logical Address} < \text{Limit Register}$.
- **Valid Translation:** If the condition holds, the physical memory address is calculated as:

$$\text{Physical Address} = \text{Base Register} + \text{Logical Address}$$

- **Invalid Access:** If $\text{Logical Address} \geq \text{Limit Register}$, the execution causes a hardware trap to the operating system (a segmentation fault or addressing error).

2. Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.

```

main.c Run
6 int main() {
12
13     printf("=== Paging Memory Management Simulation ===\n");
14     printf("Configuration: Fixed Page Size = %d Bytes\n\n", PAGE_SIZE);
15
16     printf("Displaying Page Table:\n");
17     printf("-----\n");
18     printf("Page No. | Frame No.\n");
19     printf("-----\n");
20     for (int i = 0; i < TOTAL_PAGES; i++) {
21         printf("    %d |    %d\n", i, page_table[i]);
22     }
23     printf("-----\n\n");
24
25     // Input Page Number and Offset
26     printf("Enter Logical Page Number (0-%d): ", TOTAL_PAGES - 1);
27     if (scanf("%u", &page_number) != 1) return 1;
28
29     printf("Enter Offset inside Page (0-%d): ", PAGE_SIZE - 1);
30     if (scanf("%u", &offset) != 1) return 1;
31
32     printf("\n-----\n");
33
34     // Bounds Check: Validate Page Number and Offset
35     if (page_number >= TOTAL_PAGES) {
36         printf("Status: INVALID PAGE NUMBER (Page out of logical address space bounds)\n");
37     } else if (offset >= PAGE_SIZE) {
38         printf("Status: INVALID OFFSET (Offset exceeds fixed page size limit of %d bytes)\n",
39             PAGE_SIZE);
40     } else {
41         int frame_number = page_table[page_number];
42
43         if (frame_number == -1) {
44             printf("Status: PAGE FAULT (Page %u is not loaded into any physical frame)\n",
45                 page_number);
46         } else {
47             // Compute Logical Address and Physical Address

```

```

Output
=== Paging Memory Management Simulation ===
Configuration: Fixed Page Size = 1024 Bytes

Displaying Page Table:
-----
Page No. | Frame No.
-----
0 | 5
1 | 2
2 | 9
3 | 1
4 | 4
5 | 12
6 | 0
7 | 7
-----

Enter Logical Page Number (0-7): 6
Enter Offset inside Page (0-1023): 100

-----
Status: SUCCESSFUL TRANSLATION
Logical Address : 6244 (Page 6, Offset 100)
Frame Map       : Page 6 ---> Frame 0
Physical Address Computation: (0 * 1024) + 100
Translated Physical Address : 100
-----

=== Code Execution Successful ===

```

3. Develop a program to simulate a page table. The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.

```
main.c Run
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int total_pages, page_size;
6
7     printf("=== Dynamic Page Table Translator ===\n\n");
8
9     printf("Enter total number of pages in process: ");
10    if (scanf("%d", &total_pages) != 1 || total_pages <= 0) return 1;
11
12    printf("Enter page size in bytes (e.g., 512, 1024, 4096): ");
13    if (scanf("%d", &page_size) != 1 || page_size <= 0) return 1;
14
15    int *page_table = (int *)malloc(total_pages * sizeof(int));
16
17    // Input Page Table Entries
18    printf("\n--- Page Table Configuration ---\n");
19    for (int i = 0; i < total_pages; i++) {
20        printf("Enter Frame Number for Logical Page %d (-1 if unmapped): ", i);
21        scanf("%d", &page_table[i]);
22    }
23
24    // Display Constructed Page Table
25    printf("\n===== \n");
26    printf("          CONFIGURED PAGE TABLE          \n");
27    printf("===== \n");
28    printf("Logical Page Number  -->  Physical Frame Number\n");
29    printf("----- \n");
30    for (int i = 0; i < total_pages; i++) {
31        if (page_table[i] == -1) {
32            printf("          Page %-2d      -->    [ Not Allocated ]\n", i);
33        } else {
34            printf("          Page %-2d      -->    Frame %-2d\n", i, page_table[i]);
35        }
36    }
37    printf("----- \n\n");
```

```
Output
=== Dynamic Page Table Translator ===

Enter total number of pages in process: 5
Enter page size in bytes (e.g., 512, 1024, 4096): 512

--- Page Table Configuration ---
Enter Frame Number for Logical Page 0 (-1 if unmapped): 5
Enter Frame Number for Logical Page 1 (-1 if unmapped): 2
Enter Frame Number for Logical Page 2 (-1 if unmapped): 3
Enter Frame Number for Logical Page 3 (-1 if unmapped): 6
Enter Frame Number for Logical Page 4 (-1 if unmapped): 7

=====
          CONFIGURED PAGE TABLE
=====
Logical Page Number  -->  Physical Frame Number
-----
Page 0      -->    Frame 5
Page 1      -->    Frame 2
Page 2      -->    Frame 3
Page 3      -->    Frame 6
Page 4      -->    Frame 7
-----

Enter a full Logical Address (0 to 2559): 2000

--- Translation Steps ---
1. Extract Page Number (p) = 2000 / 512 = 3
2. Extract Offset (d) = 2000 % 512 = 464
3. Page Table Lookup : Page 3 maps to Frame 6
4. Physical Address : (6 * 512) + 464 = 3536
RESULT: Logical Address 2000 --> Physical Address 3536

Translate another address? (y/n):
```

4.Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.

```
main.c Run
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 #define TLB_SIZE 4
5
6 typedef struct {
7     int page_number;
8     int frame_number;
9     bool valid;
10 } TLBEntry;
11
12 TLBEntry tlb[TLB_SIZE];
13 int fifo_counter = 0;
14
15 // Initialize TLB entries as empty
16 void init_tlb() {
17     for (int i = 0; i < TLB_SIZE; i++) {
18         tlb[i].valid = false;
19     }
20 }
21
22 // Search for page in TLB
23 int search_tlb(int page_num) {
24     for (int i = 0; i < TLB_SIZE; i++) {
25         if (tlb[i].valid && tlb[i].page_number == page_num) {
26             return tlb[i].frame_number; // TLB Hit
27         }
28     }
29     return -1; // TLB Miss
30 }
31
32 // Update TLB using FIFO replacement
33 void update_tlb(int page_num, int frame_num) {
34     tlb[fifo_counter].page_number = page_num;
35     tlb[fifo_counter].frame_number = frame_num;
36     tlb[fifo_counter].valid = true;
37 }
```

```
Output
--- Simulating Memory Access Stream ---
Access 1: Page 0 -> TLB MISS -> Fetching from Page Table (Frame 10)
Access 2: Page 1 -> TLB MISS -> Fetching from Page Table (Frame 20)
Access 3: Page 2 -> TLB MISS -> Fetching from Page Table (Frame 30)
Access 4: Page 0 -> TLB HIT (Frame 10)
Access 5: Page 3 -> TLB MISS -> Fetching from Page Table (Frame 40)
Access 6: Page 4 -> TLB MISS -> Fetching from Page Table (Frame 50)
Access 7: Page 1 -> TLB HIT (Frame 20)
Access 8: Page 5 -> TLB MISS -> Fetching from Page Table (Frame 60)
Access 9: Page 0 -> TLB MISS -> Fetching from Page Table (Frame 10)
Access 10: Page 2 -> TLB MISS -> Fetching from Page Table (Frame 30)

--- Simulation Results ---
Total Accesses : 10
TLB Hits       : 2
TLB Misses     : 8
Hit Ratio      : 20.00%
-----
EAT without TLB : 200.00 ns
EAT with TLB   : 200.00 ns
Performance Gain : 0.00%

=== Code Execution Successful ===
```

5. Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.

```
main.c Run
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 typedef struct {
5     int pid;
6     unsigned int base;
7     unsigned int limit;
8 } ProcessControlBlock;
9
10 // Function to simulate the Memory Management Unit (MMU) hardware check
11 bool access_memory(ProcessControlBlock p, unsigned int logical_addr, unsigned int *physical_addr) {
12     // Hardware Protection Check: Is 0 <= logical_addr < limit?
13     if (logical_addr < p.limit) {
14         *physical_addr = p.base + logical_addr; // Address Translation
15         return true; // Access granted
16     } else {
17         *physical_addr = 0;
18         return false; // Protection Violation / Trap to OS
19     }
20 }
21
22 int main() {
23     // Simulating a process allocated memory in RAM
24     // Base = 300040 (Start physical address), Limit = 120900 (Process size)
25     ProcessControlBlock proc1 = {
26         .pid = 101,
27         .base = 300040,
28         .limit = 120900
29     };
30
31     printf("=== Memory Protection Simulation (Base & Limit Registers) ===\n");
32     printf("Process ID : %d\n", proc1.pid);
33     printf("Base Register : %u\n", proc1.base);
34     printf("Limit Register: %u (Valid Logical Range: 0 to %u)\n", proc1.limit, proc1.limit - 1);
35     printf("Physical Range: %u to %u\n", proc1.base, proc1.base + proc1.limit - 1);
36     printf("-----\n\n");
37 }
```

```
Output
--- Simulating Memory Access Stream ---
Access 1: Page 0 -> TLB MISS -> Fetching from Page Table (Frame 10)
Access 2: Page 1 -> TLB MISS -> Fetching from Page Table (Frame 20)
Access 3: Page 2 -> TLB MISS -> Fetching from Page Table (Frame 30)
Access 4: Page 0 -> TLB HIT (Frame 10)
Access 5: Page 3 -> TLB MISS -> Fetching from Page Table (Frame 40)
Access 6: Page 4 -> TLB MISS -> Fetching from Page Table (Frame 50)
Access 7: Page 1 -> TLB HIT (Frame 20)
Access 8: Page 5 -> TLB MISS -> Fetching from Page Table (Frame 60)
Access 9: Page 0 -> TLB MISS -> Fetching from Page Table (Frame 10)
Access 10: Page 2 -> TLB MISS -> Fetching from Page Table (Frame 30)

--- Simulation Results ---
Total Accesses : 10
TLB Hits : 2
TLB Misses : 8
Hit Ratio : 20.00%
-----
EAT without TLB : 200.00 ns
EAT with TLB : 200.00 ns
Performance Gain : 0.00%

=== Code Execution Successful ===
```